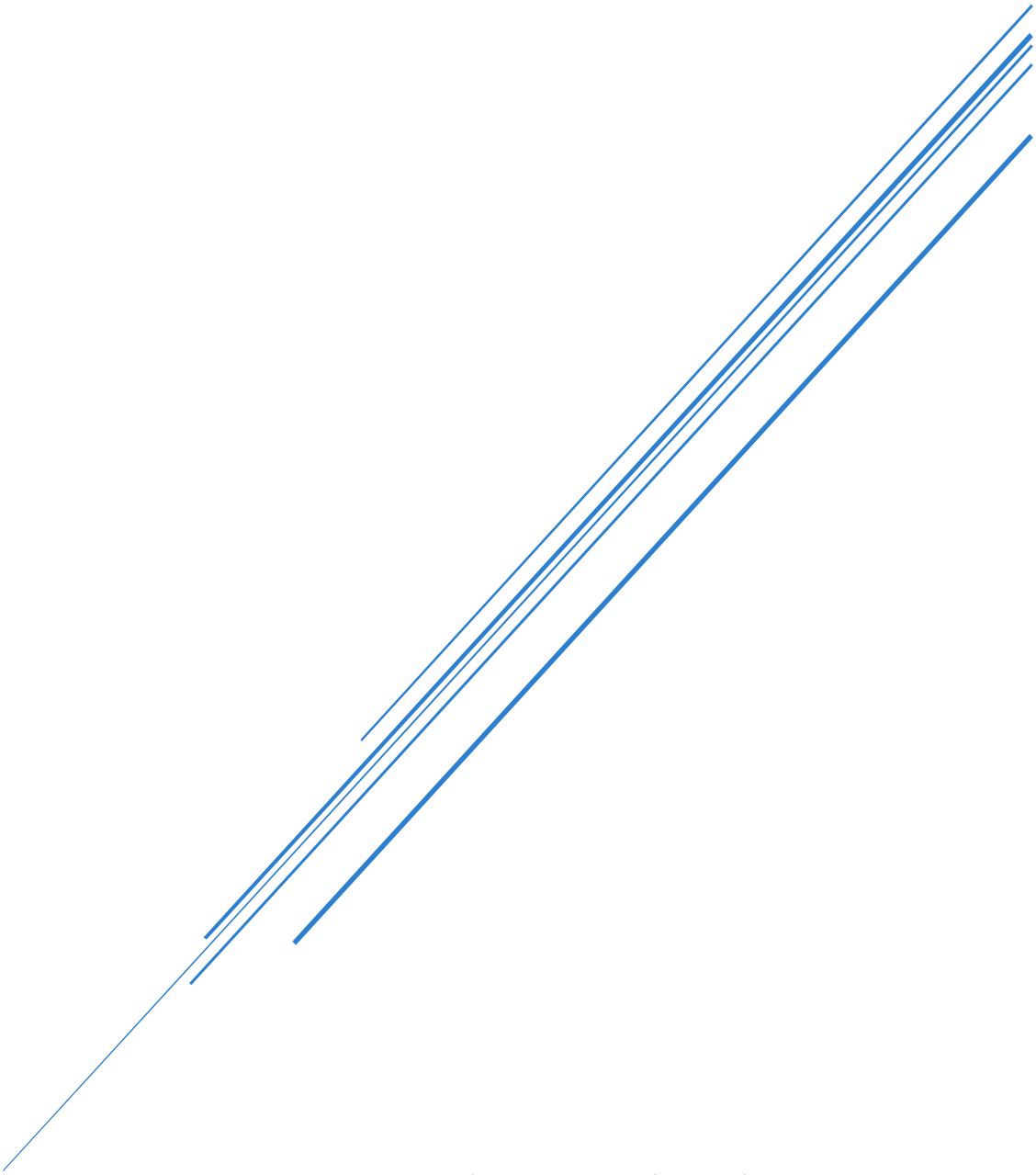
LAB 1 ANSWERS REPORT

Oliver Wuttke – WUTT0019

Date of Completion: 04/08/2026



Flinders University – Tonsley
Artificial Intelligence (COMP3742)

Table of Contents

Question 1

What are the key features of PyTorch?

Some key features of PyTorch are:

- Tensor objects.
- Precompiled CUDA functions that can be called to perform computationally expensive tasks on specialised hardware (when supported).
- Build, run, and debug neural networks.
- Contains lots of pre-built models that can be extended or used for inference.
- Computational graph model

These features make PyTorch an all-in-one library for deep learning as it supports research, development and managing production models (tensor flow is often regarded better for production).

Question 2

How do you create a tensor in PyTorch?

To create a tensor in PyTorch there're multiple ways to do so, but to explicitly define a tensor with non-random values one way is by using:

```
torch.tensor([])
```

Then you can provide your values the same way you would define a NumPy array. This allows you to create tensors of as many dimensions as you want/that your architecture can reasonably handle.

Question 3

What is the difference between element-wise multiplication and matrix multiplication in PyTorch?

Element-wise addition between two matrices (2D tensors) is also called the Hadamard product denoted as $A \odot B$ for two matrices of size $n \times m$. Elementwise multiplication requires both matrices to be of same dimensions so that the resulting matrix C is defined such as

$$\sum_{i=1}^{n}\sum_{j=1}^{m} C[i][j] = A[i][j] \times B[i][j].$$

This results in a linear time complexity of $O(nm)$ for the elementwise multiplication of two matrices of size $n \times m$.

Matrix multiplication on the other hand is a more expensive operation and has different conditions to meet. Whilst elementwise multiplication works on two matrices of the same size, matrix multiplication takes two matrices of size $N \times M$ and $M \times K$ and produces a matrix of size $N \times K$. So, for two matrices A and B of size $N \times M$ and $M \times K$ respectively the resulting matrix $C = AB$ of size $N \times K$ is defined as

$$\sum_{i=1}^{N}\sum_{k=1}^{K} C[i][k] = \sum_{j=1}^{M} A[i][j] \times B[j][k].$$

This results in a time complexity of $O(NMK)$ or for square matrices $O(N^3)$ making it a computationally more expensive operation.

Question 4

How can you perform tensor slicing and indexing in PyTorch?

For indexing the following pattern applies

```
a = torch.arange(24).reshape(2, 3, 4)

print(a[0]) # Access first 'sheet'/layer shape(3, 4)
print(a[0, 1]) # Second layer shape(4,)
print(a[0, 2, 3])# Scalar/element access for 3D tensor
```

This is a bit different to Pythons list object indexing but still somewhat intuitive.

Slicing follows a similar convention such as

```
a = torch.arange(24).reshape(2, 3, 4)

print(a[:, 1:3]) # All of dim 0, rows 1-2 of dim 1, shape(2, 2, 4)
print(a[:, :, ::2]) # All of dim 0 and dim 1 cols 0-1 of dim 2, shape(2, 3, 2)
```

Question 5

Why is it recommended to use a virtual environment when installing PyTorch?

A virtual environment is recommended because it helps keeps your project packages isolated to that project and can help with versioning issues when working on different projects on the same machine that require different versions of the same package. It's also portable and easy to use in combination with a requirement.txt file that specifies package versions.

AI Declaration

No AI was used for the exercises, lab question, or reflection.